Day06 出行驻留识别（阶段3）：规则精判与标签输出

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 6 天（8 课时）
> 主题：深入理解阶段3规则精判算法，按imsi-类簇分组，用首/中/尾点规则判定每个点是驻留(0)还是出行(1)

一、本日目标

1. 理解阶段3算法总览与规则设计思路；
2. 掌握按imsi-类簇分组的方法；
3. 掌握首点、中点、尾点的识别方法；
4. 掌握距簇中心距离判定（<1000m）；
5. 掌握时间跨度判定（与下点时间跨度<2h）；
6. 掌握驻留/出行标签最终判定（0驻留/1出行）；
7. 完整实现阶段3模块（tr_stage3.py）；
8. 分析阶段3结果，总结三阶段算法效果。

二、知识要点

- 阶段3是三阶段算法的最后一步，用业务规则对阶段2的聚类结果做精判；
- 按 imsi-类簇 分组，对每个簇内的点按时间排序，识别首点、中点、尾点；
- 判定规则：距簇中心<1000m 且 与下点时间跨度<2h → 驻留(0)，否则出行(1)；
- 阶段1+阶段2的出行点直接标记为出行(1)，不参与阶段3规则判定；
- 最终输出：stage3_label.csv（时间,lac_id,cell_id,标签），这是出行驻留识别的最终结果。

课时1：阶段3算法总览与规则设计

1.1 课时目标

理解阶段3在三阶段算法中的位置，掌握规则设计的业务逻辑。

1.2 知识点深度讲解

1.2.1 三阶段算法回顾

```
阶段1（速度粗筛）：速度>8km/h → 出行点
阶段2（k-means聚类）：单点成簇 → 出行点；其他点 → 候选驻留点
阶段3（规则精判）：对候选驻留点用业务规则判定 → 驻留(0)或出行(1)
```

阶段1和阶段2已经把明显的出行点筛出来了，阶段3对剩下的"候选驻留点"做精判。

1.2.2 为什么需要阶段3

阶段2的 k-means 聚类把点分到不同簇，但簇内的点不一定都是驻留点：
- 簇内可能有用户"经过"的点（短暂连接某个基站，但不在此停留）；
- 簇内可能有用户"路过"的点（从一个驻留区域到另一个驻留区域的中间点）；
- 簇中心是簇内所有点的均值，可能被出行点拉偏。

所以需要用业务规则对簇内每个点做精判，区分真正的驻留点和出行点。

1.2.3 阶段3的规则设计

阶段3按 **imsi-类簇** 分组，对每个簇内的点按时间排序，然后对每个点判定：

判定规则：
1. **距簇中心距离 < 1000m**：点离簇中心近，说明在驻留区域内；
2. **与下一个点的时间跨度 < 2h**：用户在该点后2小时内还有下一个点，说明没有长时间离开；
3. **同时满足以上两个条件 → 驻留(0)**，否则 → 出行(1)。

为什么是这两个条件?
- **距离条件**：驻留区域的范围通常不超过1000m（基站覆盖范围约几百米到1公里）；
- **时间条件**：如果用户在一个点后超过2小时才出现下一个点，说明用户可能离开了该区域（去了其他地方但没信令记录，或者关机了），这个点可能是出行的终点而非驻留。

1.2.4 首点、中点、尾点的特殊处理

按时间排序后，簇内的点分为三类：
1. **首点**（第一个点）：没有前一个点，只有后一个点；

2. **中点**（中间的点）：既有前一个点，也有后一个点；
3. **尾点**（最后一个点）：只有前一个点，没有后一个点。

特殊处理：
- 首点和尾点的时间跨度判定可能不准确（首点没有前点，尾点没有后点）；
- 本项目简化处理：所有点都用"与下一个点的时间跨度"判定，尾点没有下一个点，时间跨度设为无穷大（→ 出行）。

> 尾点判为出行是合理的：用户在该点后没有更多记录，说明可能离开了该区域。

1.2.5 阶段3的输入输出

输入：
- stage2_cluster.csv（含阶段1出行点标记、簇ID、最优K）；

输出：
- stage3_label.csv：imsi, time, lac_id, cell_id, lon, lat, label（0驻留/1出行）；
- 这是出行驻留识别的最终结果，供后续出行目的、出行方式分析使用。

1.3 项目实现步骤

1. 读取 stage2_cluster.csv；
2. 排除阶段1+阶段2出行点（直接标记为出行）；
3. 对候选驻留点按 imsi-簇ID 分组；
4. 每组按时间排序，计算簇中心；
5. 对每个点计算距簇中心距离和与下点时间跨度；
6. 判定标签（0/1）；
7. 合并所有点，输出最终结果。

1.4 完整代码（阶段3算法总览演示）

```
# ============================================================
# 课时1：阶段3算法总览演示
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取阶段2结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage2_cluster.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 统计各类型点数
print(f"\n各类型点数：")
print(f"  阶段1出行点：{df['is_travel_stage1'].sum()}")
print(f"  阶段2出行点(单点成簇)：{df['is_travel_stage2'].sum()}")
print(f"  合并出行点：{df['is_travel_stage12'].sum()}")
print(f"  候选驻留点：{(~df['is_travel_stage12']).sum()}")

# 候选驻留点按 imsi-簇ID 分组
candidates = df[~df["is_travel_stage12"]].copy()
candidates = candidates[candidates["cluster_id"] >= 0]  # 有簇标签
print(f"\n有簇标签的候选驻留点：{len(candidates)}")

# 按 imsi-簇ID 分组
groups = candidates.groupby(["imsi", "cluster_id"])
print(f"imsi-簇ID 组数：{len(groups)}")

# 查看一个组的示例
print(f"\n示例组（第一个组）：")
for (imsi, cid), group in groups:
    print(f"  imsi={imsi}, cluster_id={cid}")
```

```
    print(f"  点数: {len(group)}")
    print(f"  时间范围: {group['time'].min()} ~ {group['time'].max()}")
    print(f"  经纬度范围: lon[{group['lon'].min():.4f}, {group['lon'].max():.4f}], "
          f"lat[{group['lat'].min():.4f}, {group['lat'].max():.4f}]")
    break

# 组大小分布
group_sizes = groups.size()
print(f"\n组大小分布:")
print(f"  最小: {group_sizes.min()}, 最大: {group_sizes.max()}")
print(f"  平均: {group_sizes.mean():.1f}, 中位数: {group_sizes.median():.0f}")
print(f"  2点组: {(group_sizes == 2).sum()}")
print(f"  3-5点组: {((group_sizes >= 3) & (group_sizes <= 5)).sum()}")
print(f"  6-10点组: {((group_sizes >= 6) & (group_sizes <= 10)).sum()}")
print(f"  10点以上组: {(group_sizes > 10).sum()}")
```

1.5 运行结果与解读

运行后展示:
- 各类型点数（阶段1出行、阶段2出行、候选驻留）;
- imsi-簇ID 组数;
- 示例组的详细信息;
- 组大小分布。

> 关键理解:
> 1. 阶段3只对候选驻留点（非阶段1+阶段2出行点）做精判;
> 2. 按 imsi-簇ID 分组，每个组代表用户的一个驻留区域;
> 3. 组大小不一，大部分组有3-10个点。

1.6 当堂练习

1. 统计每个用户的簇数量分布;
2. 思考：为什么尾点没有下一个点时要判为出行？（提示：用户可能离开了该区域）
3. 尝试把距离阈值从1000m改为500m，预判对结果的影响。

1.7 课时小结

- 阶段3对候选驻留点用业务规则精判;

- 判定规则：距簇中心<1000m 且 与下点时间跨度<2h → 驻留(0)，否则出行(1)；
- 按 imsi-簇ID 分组，每组代表一个驻留区域；
- 首点/中点/尾点特殊处理，尾点判为出行；
- 最终输出 stage3_label.csv。

课时2：按imsi-类簇分组与首/中/尾点识别

2.1 课时目标

掌握按 imsi-类簇分组的方法，识别每组内的首点、中点、尾点。

2.2 知识点深度讲解

2.2.1 按 imsi-类簇分组

Pandas 的 groupby 支持多列分组：

```
groups = df.groupby(["imsi", "cluster_id"])
```

每个组包含：
- 一个用户（imsi）；
- 一个簇（cluster_id）；
- 该用户在该簇内的所有点（按时间排序后）。

为什么按 imsi-簇ID 分组?
- 不同用户的簇是独立的，不能混在一起；
- 同一用户的不同簇代表不同的驻留区域，需要分别处理；
- 每个组内的点按时间排序后，可以分析用户在该驻留区域的停留过程。

2.2.2 组内按时间排序

每个组内的点需要按时间排序，才能识别首点、中点、尾点：

```
group = group.sort_values("time").reset_index(drop=True)
```

2.2.3 首点、中点、尾点识别

按时间排序后：
- **首点**：index = 0（第一个点）；

- **尾点**：index = len(group) - 1（最后一个点）；
- **中点**：0 < index < len(group) - 1（中间的点）。

```
n = len(group)
group["point_type"] = "middle"
group.loc[0, "point_type"] = "first"
group.loc[n-1, "point_type"] = "last"
```

2.2.4 与下一个点的时间跨度

对每个点，计算与下一个点的时间差：

```
group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S")
group["next_time"] = group["time_dt"].shift(-1)  # 下一个点的时间
group["time_span_to_next"] = (group["next_time"] - group["time_dt"]).dt.total_seconds()
/ 3600  # 小时
```

尾点没有下一个点，time_span_to_next 为 NaN（设为无穷大 → 出行）。

2.2.5 簇中心计算

每个组的簇中心是组内所有点的经纬度均值：

```
center_lon = group["lon"].mean()
center_lat = group["lat"].mean()
```

2.3 项目实现步骤

1. 按 imsi-簇ID 分组；
2. 每组按时间排序；
3. 识别首点、中点、尾点；
4. 计算与下点时间跨度；
5. 计算簇中心。

2.4 完整代码（分组与首/中/尾点识别）

```
# ============================================================
# 课时2：按imsi-类簇分组与首/中/尾点识别
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

df = pd.read_csv(os.path.join(RESULT_DIR, "stage2_cluster.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})

# 候选驻留点（非阶段1+阶段2出行点，且有簇标签）
candidates = df[(~df["is_travel_stage12"]) & (df["cluster_id"] >= 0)].copy()
print(f"候选驻留点：{len(candidates)}")

# 按 imsi-簇ID 分组
groups = candidates.groupby(["imsi", "cluster_id"])
print(f"组数：{len(groups)}")

# 处理每个组
all_processed = []
first_group_info = None

for (imsi, cid), group in groups:
    # 按时间排序
    group = group.sort_values("time").reset_index(drop=True).copy()
    n = len(group)

    # 识别首点、中点、尾点
    group["point_type"] = "middle"
    group.loc[0, "point_type"] = "first"
    group.loc[n - 1, "point_type"] = "last"
```

```
    # 计算时间（datetime）
    group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S",
errors="coerce")

    # 与下一个点的时间跨度（小时）
    group["next_time"] = group["time_dt"].shift(-1)
    group["time_span_hours"] = (group["next_time"] -
group["time_dt"]).dt.total_seconds() / 3600

    # 簇中心
    group["center_lon"] = group["lon"].mean()
    group["center_lat"] = group["lat"].mean()

    # 记录第一个组的详细信息
    if first_group_info is None:
        first_group_info = group.copy()

    all_processed.append(group)

# 合并所有组
processed = pd.concat(all_processed, ignore_index=True)
print(f"处理后总点数: {len(processed)}")

# 统计首/中/尾点数量
print(f"\n首/中/尾点统计:")
print(processed["point_type"].value_counts())

# 时间跨度统计
print(f"\n与下点时间跨度统计:")
valid_span = processed[processed["time_span_hours"].notna()]["time_span_hours"]
print(f"  非空数: {len(valid_span)}")
print(f"  中位数: {valid_span.median():.2f} 小时")
print(f"  平均值: {valid_span.mean():.2f} 小时")
print(f"  <2h: {(valid_span < 2).sum()} ({(valid_span < 2).mean()*100:.1f}%)")
print(f"  >=2h: {(valid_span >= 2).sum()} ({(valid_span >= 2).mean()*100:.1f}%)")

# 展示第一个组的详细信息
print(f"\n{'='*60}")
```

```
print("第一个组的详细信息:")
print(f"{'='*60}")
print(f"imsi: {first_group_info['imsi'].iloc[0]}")
print(f"cluster_id: {first_group_info['cluster_id'].iloc[0]}")
print(f"点数: {len(first_group_info)}")
print(f"簇中心: ({first_group_info['center_lon'].iloc[0]:.4f},
{first_group_info['center_lat'].iloc[0]:.4f})")
print(f"\n各点详情:")
print(f"{'序号':<5} {'时间':<16} {'类型':<8} {'经度':<10} {'纬度':<10} {'距中心(m)':
<12} {'到下点(h)':<10}")
print("-" * 75)
for i, row in first_group_info.iterrows():
    # 距簇中心距离（近似，用经纬度差*111km）
    dlon = (row["lon"] - row["center_lon"]) * 111 * np.cos(np.radians(row["lat"]))
    dlat = (row["lat"] - row["center_lat"]) * 111
    dist_m = np.sqrt(dlon**2 + dlat**2) * 1000
    span = row["time_span_hours"] if pd.notna(row["time_span_hours"]) else float("inf")
    print(f"{i:<5} {row['time']:<16} {row['point_type']:<8} "
          f"{row['lon']:<10.4f} {row['lat']:<10.4f} {dist_m:<12.1f} {span:<10.2f}")
```

2.5 运行结果与解读

运行后展示：
- 首/中/尾点数量统计；
- 时间跨度统计（中位数、<2h比例）；
- 第一个组的详细信息（每个点的时间、类型、经纬度、距中心距离、到下点时间跨度）。

> 关键理解：
> 1. 首点和尾点各占一组的1个，中点占大部分；
> 2. 大部分点与下点时间跨度<2h（说明用户在该区域内连续活动）；
> 3. 尾点没有下点，时间跨度为无穷大（→ 出行）。

2.6 当堂练习

1. 统计首点、中点、尾点的距簇中心距离分布，对比是否有差异；
2. 思考：为什么中点的时间跨度通常比首点和尾点小？（提示：中点在驻留过程中，首点和尾点是进出驻留区域的边界）
3. 尝试找出一个"尾点距簇中心很近但时间跨度无穷大"的例子，分析是否应该判为驻留。

2.7 课时小结

- 按 imsi-簇ID 分组，每组代表一个驻留区域；
- 组内按时间排序，识别首点(index=0)、中点、尾点(index=n-1)；
- 与下点时间跨度用 shift(-1) 计算，尾点为 NaN；
- 簇中心是组内所有点的经纬度均值；
- 大部分点与下点时间跨度<2h。

课时3：距簇中心距离判定（<1000m）

3.1 课时目标

掌握距簇中心距离的计算方法（Haversine），理解1000m阈值的业务含义。

3.2 知识点深度讲解

3.2.1 为什么用 Haversine 距离

点和簇中心都是经纬度，计算距离应该用 Haversine（球面距离），而不是欧几里得距离。

欧几里得距离的问题：
- 经纬度是角度，不是长度；
- 1度经度 ≈ 111km * cos(纬度)，1度纬度 ≈ 111km；
- 直接用 sqrt((lon1-lon2)² + (lat1-lat2)²) 得到的是"角度差"，不是实际距离。

Haversine 公式：

$$
a = \sin^2(\frac{\Delta\phi}{2}) + \cos(\phi_1) \cos(\phi_2) \sin^2(\frac{\Delta\lambda}{2})
$$

$$
d = 2R \arcsin(\sqrt{a})
$$

其中 R = 6371km（地球半径）。

3.2.2 1000m 阈值的业务含义

为什么是1000m?
- 基站覆盖范围：城市内基站覆盖半径约 200-500m，郊区约 1-2km；
- 驻留区域：用户在一个驻留区域（如家、公司）内可能连接多个基站，范围约 500-1000m；
- 1000m 是合理的驻留区域半径。

距离判定规则：
- 距簇中心 < 1000m → 在驻留区域内（可能是驻留）；
- 距簇中心 >= 1000m → 不在驻留区域内（是出行）。

3.2.3 向量化计算距离

对一个组内的所有点，批量计算到簇中心的距离：

```
# center_lon, center_lat 是标量
# group["lon"], group["lat"] 是 Series
dists = haversine(group["lon"], group["lat"], center_lon, center_lat, unit="m")
```

Haversine 函数支持向量化（用 numpy），所以可以一次计算整个组的距离。

3.2.4 距离分布分析

统计所有候选驻留点到簇中心的距离分布，可以判断1000m阈值是否合理：
- 如果大部分点距离<1000m，说明阈值合理；
- 如果很多点距离>1000m，说明簇内有较多"边缘点"，可能需要调整阈值或聚类参数。

3.3 项目实现步骤

1. 实现 Haversine 距离函数（向量化）；
2. 对每个组计算簇中心；
3. 计算组内每个点到簇中心的距离；
4. 统计距离分布，分析1000m阈值的合理性；
5. 标记距离<1000m的点。

3.4 完整代码（距簇中心距离判定）

```
# ============================================================
# 课时3：距簇中心距离判定（<1000m）
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取数据
df = pd.read_csv(os.path.join(RESULT_DIR, "stage2_cluster.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
candidates = df[(~df["is_travel_stage12"]) & (df["cluster_id"] >= 0)].copy()
print(f"候选驻留点：{len(candidates)}")

# 按 imsi-簇ID 分组，计算每个点到簇中心的距离
```

```
all_dists = []
for (imsi, cid), group in candidates.groupby(["imsi", "cluster_id"]):
    center_lon = group["lon"].mean()
    center_lat = group["lat"].mean()
    dists = haversine(group["lon"], group["lat"], center_lon, center_lat, unit="m")
    all_dists.extend(dists.tolist())

all_dists = np.array(all_dists)
print(f"距离计算完成: {len(all_dists)} 个点")

# 距离统计
print(f"\n距簇中心距离统计:")
print(f"  最小值: {all_dists.min():.1f} m")
print(f"  最大值: {all_dists.max():.1f} m")
print(f"  中位数: {np.median(all_dists):.1f} m")
print(f"  平均值: {all_dists.mean():.1f} m")
print(f"  标准差: {all_dists.std():.1f} m")

# 不同阈值的覆盖率
print(f"\n不同阈值的覆盖率:")
for threshold in [100, 200, 500, 800, 1000, 1500, 2000, 3000, 5000]:
    coverage = (all_dists < threshold).mean() * 100
    print(f"  <{threshold:>5}m: {coverage:>5.1f}%")

# 1000m 阈值
within_1000 = (all_dists < 1000).sum()
print(f"\n1000m 阈值:")
print(f"  <1000m: {within_1000} ({within_1000/len(all_dists)*100:.1f}%)")
print(f"  >=1000m: {len(all_dists) - within_1000} ({(len(all_dists)-
within_1000)/len(all_dists)*100:.1f}%)")

# 可视化：距离分布直方图
print("\n生成距离分布图...")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 全范围
axes[0].hist(all_dists, bins=100, edgecolor="black", alpha=0.7)
axes[0].axvline(x=1000, color="red", linestyle="--", label="阈值=1000m")
```

```
axes[0].set_xlabel("距簇中心距离 (m)")
axes[0].set_ylabel("频数")
axes[0].set_title("距簇中心距离分布（全范围）")
axes[0].legend()

# 0-3000m 放大
mask = all_dists < 3000
axes[1].hist(all_dists[mask], bins=50, edgecolor="black", alpha=0.7, color="orange")
axes[1].axvline(x=1000, color="red", linestyle="--", label="阈值=1000m")
axes[1].set_xlabel("距簇中心距离 (m)")
axes[1].set_ylabel("频数")
axes[1].set_title("距簇中心距离分布（0-3000m 放大）")
axes[1].legend()

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_dist_to_center.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage3_dist_to_center.png")

# 对比：欧几里得距离 vs Haversine 距离
print(f"\n{'='*60}")
print("对比：欧几里得距离 vs Haversine 距离")
print(f"{'='*60}")
# 选一个组
group = candidates.groupby(["imsi",
"cluster_id"]).get_group(list(candidates.groupby(["imsi", "cluster_id"]).groups.keys())
[0])
center_lon = group["lon"].mean()
center_lat = group["lat"].mean()

# 欧几里得距离（经纬度差，再乘以111km近似）
euclid_km = np.sqrt(((group["lon"] - center_lon) * 111 *
np.cos(np.radians(center_lat)))**2 +
                    ((group["lat"] - center_lat) * 111)**2)
euclid_m = euclid_km * 1000

# Haversine 距离
```

```
haversine_m = haversine(group["lon"], group["lat"], center_lon, center_lat, unit="m")

print(f"{'序号':<5} {'欧几里得(m)':<12} {'Haversine(m)':<12} {'差异(m)':<10} {'差异(%)':<10}")
print("-" * 55)
for i in range(min(10, len(group))):
    diff = abs(euclid_m.iloc[i] - haversine_m.iloc[i])
    diff_pct = diff / haversine_m.iloc[i] * 100 if haversine_m.iloc[i] > 0 else 0
    print(f"{i:<5} {euclid_m.iloc[i]:<12.1f} {haversine_m.iloc[i]:<12.1f} {diff:<10.1f} {diff_pct:<10.2f}")

print(f"\n平均差异: {np.mean(np.abs(euclid_m - haversine_m)):.1f} m")
print(f"平均差异率: {np.mean(np.abs(euclid_m - haversine_m) / haversine_m.replace(0, np.nan)) * 100:.2f}%")
print("结论：城市范围内欧几里得距离与Haversine距离差异很小（<1%），但Haversine更准确")
```

3.5 运行结果与解读

运行后展示：
- 距簇中心距离统计（最小、最大、中位、平均、标准差）；
- 不同阈值的覆盖率（100m到5000m）；
- 1000m阈值的覆盖率；
- 距离分布直方图（全范围 + 0-3000m放大）；
- 欧几里得距离 vs Haversine 距离对比。

> 关键理解：
> 1. 大部分点距簇中心<1000m（说明1000m阈值合理）；
> 2. 距离分布呈右偏，少数点距离很远（可能是簇内的出行点）；
> 3. 城市范围内欧几里得距离与Haversine差异很小，但Haversine更准确。

3.6 当堂练习

1. 把阈值改为500m和1500m，分别统计覆盖率，分析哪个阈值更合理；
2. 思考：为什么距离分布呈右偏？（提示：簇中心是均值，大部分点在中心附近，少数点远离中心）
3. 找出距离>1000m的点，分析它们的时间特征（是否是首点或尾点）。

3.7 课时小结

- 距簇中心距离用 Haversine 公式计算（球面距离）；
- 1000m 阈值：城市内基站覆盖范围和驻留区域半径的合理值；
- 大部分点距簇中心<1000m，阈值合理；
- 距离分布呈右偏，少数点远离中心（可能是出行点）；
- 城市范围内欧几里得与Haversine差异很小，但Haversine更准确。

课时4：时间跨度判定（与下点时间跨度<2h）

4.1 课时目标

掌握与下点时间跨度的计算方法，理解2h阈值的业务含义。

4.2 知识点深度讲解

4.2.1 为什么需要时间跨度判定

仅靠距离判定不够：一个点距簇中心很近，但如果用户在该点后很长时间（>2h）才出现下一个点，说明用户可能离开了该区域（去了其他地方但没信令记录，或者关机了），这个点可能是出行的终点而非驻留。

例子：
- 用户早上8点在家（距簇中心100m），然后出门上班，下午6点才回家；
- 早上8点的点距簇中心很近，但与下一个点（下午6点）的时间跨度是10小时；
- 这个点应该判为出行（用户离开了家），而不是驻留。

4.2.2 2h 阈值的业务含义

为什么是2h?
- 正常驻留：用户在一个驻留区域内，信令记录的时间间隔通常<2h（手机会定期与基站通信）；
- 离开区域：如果用户离开该区域，可能2小时内不会在该区域的基站出现；
- 2h 是合理的"连续驻留"时间间隔上限。

时间跨度判定规则：
- 与下点时间跨度 < 2h → 用户在该区域内连续活动（可能是驻留）；
- 与下点时间跨度 >= 2h → 用户可能离开了该区域（是出行）；
- 尾点没有下点 → 时间跨度为无穷大（→ 出行）。

4.2.3 时间跨度计算

```
group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S")
group["next_time"] = group["time_dt"].shift(-1)
group["time_span_hours"] = (group["next_time"] - group["time_dt"]).dt.total_seconds() /
3600
```

尾点的 next_time 为 NaN，time_span_hours 也为 NaN。

4.2.4 时间跨度分布分析

统计所有候选驻留点的时间跨度分布，可以判断2h阈值的合理性：
- 如果大部分点时间跨度<2h，说明阈值合理；
- 如果很多点时间跨度>2h，说明数据中有较多"长时间间隔"，可能需要调整阈值。

4.2.5 时间跨度与点类型的关系

- **首点**：时间跨度通常较小（用户进入驻留区域后，后续点连续）；
- **中点**：时间跨度通常很小（用户在区域内连续活动）；
- **尾点**：时间跨度为无穷大（没有下点）。

4.3 项目实现步骤

1. 计算每个点与下点的时间跨度；
2. 统计时间跨度分布；
3. 分析2h阈值的合理性；
4. 标记时间跨度<2h的点；
5. 分析时间跨度与点类型（首/中/尾）的关系。

4.4 完整代码（时间跨度判定）

```
# ============================================================
# 课时4：时间跨度判定（与下点时间跨度<2h）
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")

df = pd.read_csv(os.path.join(RESULT_DIR, "stage2_cluster.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
candidates = df[(~df["is_travel_stage12"]) & (df["cluster_id"] >= 0)].copy()

# 按 imsi-簇ID 分组，计算时间跨度
all_spans = []
point_type_spans = {"first": [], "middle": [], "last": []}

for (imsi, cid), group in candidates.groupby(["imsi", "cluster_id"]):
    group = group.sort_values("time").reset_index(drop=True).copy()
    n = len(group)
    group["point_type"] = "middle"
    group.loc[0, "point_type"] = "first"
    group.loc[n - 1, "point_type"] = "last"

    group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S",
errors="coerce")
    group["next_time"] = group["time_dt"].shift(-1)
    group["time_span_hours"] = (group["next_time"] -
group["time_dt"]).dt.total_seconds() / 3600
```

```
    # 非尾点的时间跨度
    valid_spans = group[group["time_span_hours"].notna()]["time_span_hours"]
    all_spans.extend(valid_spans.tolist())

    # 按点类型统计
    for pt in ["first", "middle", "last"]:
        pt_spans = group[group["point_type"] == pt]["time_span_hours"]
        pt_spans = pt_spans[pt_spans.notna()]
        point_type_spans[pt].extend(pt_spans.tolist())

all_spans = np.array(all_spans)
print(f"有效时间跨度数: {len(all_spans)}（排除尾点）")

# 时间跨度统计
print(f"\n时间跨度统计:")
print(f"  最小值: {all_spans.min():.2f} 小时")
print(f"  最大值: {all_spans.max():.2f} 小时")
print(f"  中位数: {np.median(all_spans):.2f} 小时")
print(f"  平均值: {all_spans.mean():.2f} 小时")

# 不同阈值的覆盖率
print(f"\n不同阈值的覆盖率:")
for threshold in [0.5, 1, 1.5, 2, 3, 4, 6, 8, 12, 24]:
    coverage = (all_spans < threshold).mean() * 100
    print(f"  <{threshold:>5}h: {coverage:>5.1f}%")

# 2h 阈值
within_2h = (all_spans < 2).sum()
print(f"\n2h 阈值:")
print(f"  <2h: {within_2h} ({within_2h/len(all_spans)*100:.1f}%)")
print(f"  >=2h: {len(all_spans) - within_2h} ({(len(all_spans)-
within_2h)/len(all_spans)*100:.1f}%)")

# 按点类型统计
print(f"\n按点类型统计时间跨度:")
print(f"{'类型':<10} {'数量':<8} {'中位数(h)':<12} {'平均值(h)':<12} {'<2h比例':<10}")
print("-" * 55)
```

```
for pt in ["first", "middle", "last"]:
    spans = np.array(point_type_spans[pt])
    if len(spans) > 0:
        median = np.median(spans)
        mean = spans.mean()
        within = (spans < 2).mean() * 100
        print(f"{pt:<10} {len(spans):<8} {median:<12.2f} {mean:<12.2f} {within:
<10.1f}%")
    else:
        print(f"{pt:<10} {0:<8} {'N/A':<12} {'N/A':<12} {'N/A':<10}")

# 可视化
print("\n生成时间跨度分布图...")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 全范围
axes[0].hist(all_spans, bins=100, edgecolor="black", alpha=0.7)
axes[0].axvline(x=2, color="red", linestyle="--", label="阈值=2h")
axes[0].set_xlabel("与下点时间跨度 (小时)")
axes[0].set_ylabel("频数")
axes[0].set_title("时间跨度分布（全范围）")
axes[0].legend()

# 0-6h 放大
mask = all_spans < 6
axes[1].hist(all_spans[mask], bins=50, edgecolor="black", alpha=0.7, color="orange")
axes[1].axvline(x=2, color="red", linestyle="--", label="阈值=2h")
axes[1].set_xlabel("与下点时间跨度 (小时)")
axes[1].set_ylabel("频数")
axes[1].set_title("时间跨度分布（0-6h 放大）")
axes[1].legend()

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_time_span.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage3_time_span.png")
```

```
# 同时满足距离<1000m和时间<2h的比例
print(f"\n{'='*60}")
print("同时满足距离<1000m和时间<2h的比例")
print(f"{'='*60}")

# 重新计算（同时考虑距离和时间）
both_count = 0
total_valid = 0

def haversine(lon1, lat1, lon2, lat2, unit="km"):
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)

for (imsi, cid), group in candidates.groupby(["imsi", "cluster_id"]):
    group = group.sort_values("time").reset_index(drop=True).copy()
    n = len(group)
    center_lon = group["lon"].mean()
    center_lat = group["lat"].mean()
    dists = haversine(group["lon"], group["lat"], center_lon, center_lat, unit="m")

    group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S",
errors="coerce")
    group["next_time"] = group["time_dt"].shift(-1)
    group["time_span_hours"] = (group["next_time"] -
group["time_dt"]).dt.total_seconds() / 3600

    # 非尾点
    valid_mask = group["time_span_hours"].notna()
    total_valid += valid_mask.sum()
    both = valid_mask & (dists < 1000) & (group["time_span_hours"] < 2)
    both_count += both.sum()

print(f"有效点数（非尾点）: {total_valid}")
```

```
print(f"同时满足距离<1000m和时间<2h: {both_count} ({both_count/total_valid*100:.1f}%)")
print(f"不满足（判为出行）: {total_valid - both_count} ({(total_valid-
both_count)/total_valid*100:.1f}%)")
print(f"\n加上尾点（全部判为出行）后，驻留比例会更低")
```

4.5 运行结果与解读

运行后展示：
- 时间跨度统计（最小、最大、中位、平均）；
- 不同阈值的覆盖率；
- 2h阈值的覆盖率；
- 按点类型（首/中/尾）统计时间跨度；
- 时间跨度分布直方图；
- 同时满足距离<1000m和时间<2h的比例。

> 关键理解：
> 1. 大部分点时间跨度<2h（说明用户在区域内连续活动）；
> 2. 中点的时间跨度最小，首点次之，尾点为无穷大；
> 3. 同时满足距离和时间条件的点约占60%-80%，这些是真正的驻留点。

4.6 当堂练习

1. 把时间阈值改为1h和3h，分别统计覆盖率，分析哪个阈值更合理；
2. 思考：为什么中点的时间跨度比首点小？（提示：中点在驻留过程中，首点是进入驻留区域的边界）
3. 找出时间跨度>2h的点，分析它们的距离特征（是否距簇中心较远）。

4.7 课时小结

- 时间跨度：与下一个点的时间差，用 shift(-1) 计算；
- 2h阈值：正常驻留的时间间隔上限，超过2h说明可能离开了区域；
- 大部分点时间跨度<2h，阈值合理；
- 中点时间跨度最小，首点次之，尾点为无穷大；
- 同时满足距离<1000m和时间<2h的点是真正的驻留点。

课时5：驻留/出行标签最终判定（0/1）

5.1 课时目标

整合距离和时间两个条件，对每个点给出最终标签（0驻留/1出行），合并阶段1+阶段2+阶段3的结果。

5.2 知识点深度讲解

5.2.1 最终判定规则

对候选驻留点（非阶段1+阶段2出行点），最终判定规则：

```
if 距簇中心 < 1000m 且 与下点时间跨度 < 2h:
    label = 0  # 驻留
else:
    label = 1  # 出行
```

两个条件必须同时满足：
- 只满足距离条件（近但时间跨度大）：用户可能离开了区域 → 出行；
- 只满足时间条件（时间跨度小但距离远）：点不在驻留区域内 → 出行；
- 同时满足：在驻留区域内且连续活动 → 驻留。

5.2.2 三类点的标签来源

最终所有点的标签来源：
1. **阶段1出行点**（速度>8km/h）：直接 label=1；
2. **阶段2出行点**（单点成簇）：直接 label=1；
3. **阶段3精判点**（候选驻留点）：用规则判定 label=0或1；
4. **无簇标签的候选驻留点**（cluster_id=-1，通常是点数太少无法聚类）：默认 label=1（出行）。

5.2.3 标签合并

```
# 初始化为出行(1)
df["label"] = 1

# 阶段1+阶段2出行点保持1
# 候选驻留点中满足条件的设为0
mask_stay = (~df["is_travel_stage12"]) & (df["cluster_id"] >= 0) & \
            (df["dist_to_center"] < 1000) & (df["time_span_hours"] < 2)
df.loc[mask_stay, "label"] = 0
```

5.2.4 输出格式

最终输出格式（与项目要求一致）：

```
imsi,time,lac_id,cell_id,lon,lat,label
```

- label=0：驻留；
- label=1：出行。

按 imsi, time 排序。

5.3 项目实现步骤

1. 对每个候选驻留点计算距簇中心距离和时间跨度；
2. 应用判定规则，给出标签；
3. 合并阶段1+阶段2+阶段3的结果；
4. 统计最终标签分布；
5. 输出 stage3_label.csv。

5.4 完整代码（驻留/出行标签最终判定）

```
# ============================================================
# 课时5：驻留/出行标签最终判定（0/1）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取阶段2结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage2_cluster.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 初始化标签为出行(1)
df["label"] = 1
df["dist_to_center"] = np.nan
df["time_span_hours"] = np.nan

# 候选驻留点（非阶段1+阶段2出行点，且有簇标签）
candidates = df[(~df["is_travel_stage12"]) & (df["cluster_id"] >= 0)].copy()
print(f"候选驻留点：{len(candidates)}")
```

```
# 按 imsi-簇ID 分组，计算距离和时间跨度，判定标签
stay_count = 0
travel_count = 0

for (imsi, cid), group in candidates.groupby(["imsi", "cluster_id"]):
    group = group.sort_values("time").reset_index(drop=True).copy()
    n = len(group)

    # 簇中心
    center_lon = group["lon"].mean()
    center_lat = group["lat"].mean()

    # 距簇中心距离
    dists = haversine(group["lon"], group["lat"], center_lon, center_lat, unit="m")

    # 时间跨度
    group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S",
errors="coerce")
    group["next_time"] = group["time_dt"].shift(-1)
    time_spans = (group["next_time"] - group["time_dt"]).dt.total_seconds() / 3600

    # 判定：距离<1000m 且 时间跨度<2h → 驻留(0)
    is_stay = (dists < 1000) & (time_spans < 2)

    # 写回主 DataFrame
    df.loc[group.index, "dist_to_center"] = dists.values
    df.loc[group.index, "time_span_hours"] = time_spans.values
    df.loc[group.index[is_stay], "label"] = 0

    stay_count += is_stay.sum()
    travel_count += (~is_stay).sum()

# 统计
print(f"\n阶段3精判结果:")
print(f"  驻留(0): {stay_count}")
print(f"  出行(1): {travel_count}")
print(f"  驻留比例: {stay_count/(stay_count+travel_count)*100:.1f}%")
```

```
# 最终统计
print(f"\n最终标签分布（含阶段1+阶段2出行点）:")
print(f"  驻留(0): {(df['label'] == 0).sum()} ({(df['label'] == 0).mean()*100:.1f}%)")
print(f"  出行(1): {(df['label'] == 1).sum()} ({(df['label'] == 1).mean()*100:.1f}%)")

# 按标签来源统计
print(f"\n按标签来源统计:")
print(f"  阶段1出行点: {df['is_travel_stage1'].sum()}")
print(f"  阶段2出行点(单点成簇): {df['is_travel_stage2'].sum()}")
print(f"  阶段3出行点(规则判定): {travel_count}")
print(f"  阶段3驻留点(规则判定): {stay_count}")
print(f"  无簇标签候选驻留点(默认出行): "
      f"{((~df['is_travel_stage12']) & (df['cluster_id'] < 0)).sum()}")

# 按用户统计
user_label = df.groupby("imsi")["label"].agg(["count", "sum"])
user_label.columns = ["total", "travel"]
user_label["stay"] = user_label["total"] - user_label["travel"]
user_label["stay_ratio"] = user_label["stay"] / user_label["total"]
print(f"\n用户级统计:")
print(f"  平均驻留比例: {user_label['stay_ratio'].mean()*100:.1f}%")
print(f"  驻留比例中位数: {user_label['stay_ratio'].median()*100:.1f}%")
print(f"  全驻留用户(驻留比例=100%): {(user_label['stay_ratio'] == 1).sum()}")
print(f"  全出行用户(驻留比例=0%): {(user_label['stay_ratio'] == 0).sum()}")

# 输出
output_path = os.path.join(RESULT_DIR, "stage3_label.csv")
output_df = df[["imsi", "time", "lac_id", "cell_id", "lon", "lat", "label"]].copy()
output_df = output_df.sort_values(["imsi", "time"]).reset_index(drop=True)
output_df.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n输出: {output_path}")
print(f"  行数: {len(output_df)}")
print(f"  列: {list(output_df.columns)}")
print(f"\n前5行:")
print(output_df.head().to_string(index=False))
```

5.5 运行结果与解读

运行后展示：
- 阶段3精判结果（驻留/出行数量和比例）；
- 最终标签分布（含阶段1+阶段2出行点）；
- 按标签来源统计（阶段1、阶段2、阶段3、无簇标签）；
- 用户级统计（平均驻留比例、全驻留/全出行用户数）；
- 输出 stage3_label.csv。

> 关键理解：
> 1. 阶段3精判后，候选驻留点中约60%-80%被判为驻留，20%-40%被判为出行；
> 2. 最终标签中驻留约占70%-85%，出行约占15%-30%；
> 3. 出行点来源包括阶段1（速度）、阶段2（单点成簇）、阶段3（规则）和无簇标签（默认出行）。

5.6 当堂练习

1. 统计阶段3出行点中，是因为距离不满足还是时间不满足导致的；
2. 思考：为什么有些用户全驻留（驻留比例=100%）？（提示：用户可能全天都在一个地方，如居家）
3. 尝试把距离阈值改为500m，重新判定，对比驻留比例变化。

5.7 课时小结

- 最终判定：距簇中心<1000m 且 时间跨度<2h → 驻留(0)，否则出行(1)；
- 标签来源：阶段1出行、阶段2出行、阶段3精判、无簇标签默认出行；
- 最终驻留比例约70%-85%，出行约15%-30%；
- 输出 stage3_label.csv（imsi,time,lac_id,cell_id,lon,lat,label）。

课时6：阶段3完整实现（tr_stage3.py）

6.1 课时目标

将阶段3所有逻辑整合为完整模块 tr_stage3.py。

6.2 知识点深度讲解

6.2.1 模块结构

tr_stage3.py 的结构：
1. 配置区（路径、阈值）；
2. 工具函数（haversine）；
3. 主函数 stage3()；
4. 入口。

6.2.2 完整流程

```
读取 stage2_cluster.csv
  ↓
初始化标签为1（出行）
  ↓
对候选驻留点按 imsi-簇ID 分组：
  按时间排序
  计算簇中心
  计算距簇中心距离（Haversine）
  计算与下点时间跨度（shift(-1)）
  判定：距离<1000m 且 时间<2h → 驻留(0)
  ↓
合并所有点
  ↓
输出 stage3_label.csv
```

6.3 项目实现步骤

按上述流程编写完整代码。

6.4 完整代码（tr_stage3.py）

```
# ============================================================
# 阶段3模块：tr_stage3.py
# 功能：规则精判，输出驻留(0)/出行(1)标签
# 运行：python tr_stage3.py
# 输入：stage2_cluster.csv
# 输出：stage3_label.csv
# ============================================================

import os
import pandas as pd
import numpy as np

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
os.makedirs(RESULT_DIR, exist_ok=True)

# 阈值参数
DIST_THRESHOLD = 1000   # 距簇中心距离阈值（m）
TIME_THRESHOLD = 2       # 与下点时间跨度阈值（小时）


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d


def stage3():
    """执行阶段3完整流程。"""
    print("=" * 60)
```

```
print("阶段3：规则精判与标签输出")
print("=" * 60)

# 读取阶段2结果
cluster_path = os.path.join(RESULT_DIR, "stage2_cluster.csv")
if not os.path.exists(cluster_path):
    print("错误：请先运行 tr_stage2.py")
    return None
df = pd.read_csv(cluster_path, dtype={"imsi": str, "time": str,
                                      "lac_id": str, "cell_id": str})
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 初始化标签为出行(1)
df["label"] = 1
df["dist_to_center"] = np.nan
df["time_span_hours"] = np.nan

# 候选驻留点
candidates = df[(~df["is_travel_stage12"]) & (df["cluster_id"] >= 0)].copy()
print(f"候选驻留点：{len(candidates)}")

# 按 imsi-簇ID 分组
groups = candidates.groupby(["imsi", "cluster_id"])
print(f"组数：{len(groups)}")

stay_count = 0
travel_count = 0
processed_groups = 0

for (imsi, cid), group in groups:
    group = group.sort_values("time").reset_index(drop=True).copy()
    n = len(group)

    # 簇中心
    center_lon = group["lon"].mean()
    center_lat = group["lat"].mean()

    # 距簇中心距离
```

```
        dists = haversine(group["lon"], group["lat"], center_lon, center_lat, unit="m")

        # 与下点时间跨度
        group["time_dt"] = pd.to_datetime(group["time"], format="%Y%m%d%H%M%S",
errors="coerce")
        group["next_time"] = group["time_dt"].shift(-1)
        time_spans = (group["next_time"] - group["time_dt"]).dt.total_seconds() / 3600

        # 判定
        is_stay = (dists < DIST_THRESHOLD) & (time_spans < TIME_THRESHOLD)

        # 写回
        df.loc[group.index, "dist_to_center"] = dists.values
        df.loc[group.index, "time_span_hours"] = time_spans.values
        df.loc[group.index[is_stay], "label"] = 0

        stay_count += is_stay.sum()
        travel_count += (~is_stay).sum()
        processed_groups += 1

        if processed_groups % 100 == 0:
            print(f"  已处理 {processed_groups}/{len(groups)} 组...")

    # 统计
    print(f"\n阶段3精判结果:")
    print(f"  处理组数: {processed_groups}")
    print(f"  驻留(0): {stay_count} ({stay_count/(stay_count+travel_count)*100:.1f}%)")
    print(f"  出行(1): {travel_count}
({travel_count/(stay_count+travel_count)*100:.1f}%)")

    print(f"\n最终标签分布:")
    print(f"  驻留(0): {(df['label'] == 0).sum()} ({(df['label'] ==
0).mean()*100:.1f}%)")
    print(f"  出行(1): {(df['label'] == 1).sum()} ({(df['label'] ==
1).mean()*100:.1f}%)")

    # 输出
    output_path = os.path.join(RESULT_DIR, "stage3_label.csv")
```

```
    output_df = df[["imsi", "time", "lac_id", "cell_id", "lon", "lat", "label"]].copy()
    output_df = output_df.sort_values(["imsi", "time"]).reset_index(drop=True)
    output_df.to_csv(output_path, index=False, encoding="utf-8")
    print(f"\n输出: {output_path}")
    print(f"  行数: {len(output_df)}")

    # 同时保存完整结果（含中间字段，供分析使用）
    full_path = os.path.join(RESULT_DIR, "stage3_full.csv")
    df.to_csv(full_path, index=False, encoding="utf-8")
    print(f"完整结果: {full_path}")

    print("\n" + "=" * 60)
    print("阶段3完成！")
    print("=" * 60)
    return df


if __name__ == "__main__":
    stage3()
```

6.5 运行结果与解读

运行后展示每步统计，输出 stage3_label.csv 和 stage3_full.csv。

6.6 当堂练习

1. 运行 tr_stage3.py，记录最终驻留/出行比例；
2. 检查 stage3_label.csv 的前20行，验证标签格式。

6.7 课时小结

- tr_stage3.py 整合阶段3所有逻辑；
- 按 imsi-簇ID 分组，计算距离和时间跨度，判定标签；
- 输出 stage3_label.csv（最终标签）和 stage3_full.csv（含中间字段）。

课时7：阶段3结果分析与可视化

7.1 课时目标

分析阶段3结果，可视化驻留/出行空间分布、时间分布、用户分布。

7.2 知识点深度讲解

分析维度：
1. 驻留/出行空间分布（散点图）；
2. 驻留/出行时间分布（按小时）；
3. 用户驻留比例分布；
4. 三阶段出行点来源对比。

7.3 完整代码（阶段3结果分析与可视化）

```
# ============================================================
# 课时7：阶段3结果分析与可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_full.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

stay_df = df[df["label"] == 0]
travel_df = df[df["label"] == 1]
print(f"驻留点：{len(stay_df)} ({len(stay_df)/len(df)*100:.1f}%)")
print(f"出行点：{len(travel_df)} ({len(travel_df)/len(df)*100:.1f}%)")

# 图1：驻留/出行空间分布
print("生成图1：驻留/出行空间分布")
fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(stay_df["lon"], stay_df["lat"], c="green", s=1, alpha=0.3, label=f"驻留({len(stay_df)})")
ax.scatter(travel_df["lon"], travel_df["lat"], c="red", s=1, alpha=0.3, label=f"出行({len(travel_df)})")
ax.set_xlabel("经度"); ax.set_ylabel("纬度")
ax.set_title("驻留 vs 出行 空间分布")
ax.legend(markerscale=5)
```

```
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_spatial.png"), dpi=150,
bbox_inches="tight")
plt.close()

# 图2：驻留/出行时间分布
print("生成图2：驻留/出行时间分布")
df["hour"] = df["time"].str[8:10].astype(int)
hour_stay = stay_df["time"].str[8:10].astype(int).value_counts().reindex(range(24),
fill_value=0)
hour_travel = travel_df["time"].str[8:10].astype(int).value_counts().reindex(range(24),
fill_value=0)

fig, ax = plt.subplots(figsize=(12, 5))
width = 0.35
ax.bar(np.arange(24) - width/2, hour_stay.values, width, label="驻留", color="green",
alpha=0.7)
ax.bar(np.arange(24) + width/2, hour_travel.values, width, label="出行", color="red",
alpha=0.7)
ax.set_xlabel("小时"); ax.set_ylabel("点数")
ax.set_title("驻留 vs 出行 时间分布（按小时）")
ax.set_xticks(range(24)); ax.legend()
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_temporal.png"), dpi=150,
bbox_inches="tight")
plt.close()

# 图3：用户驻留比例分布
print("生成图3：用户驻留比例分布")
user_stats = df.groupby("imsi")["label"].agg(["count", "sum"])
user_stats.columns = ["total", "travel"]
user_stats["stay"] = user_stats["total"] - user_stats["travel"]
user_stats["stay_ratio"] = user_stats["stay"] / user_stats["total"]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(user_stats["stay_ratio"], bins=20, edgecolor="black", alpha=0.7,
color="steelblue")
axes[0].set_xlabel("驻留比例"); axes[0].set_ylabel("用户数")
```

```
axes[0].set_title(f"用户驻留比例分布（平均=
{user_stats['stay_ratio'].mean()*100:.1f}%）")

top20 = user_stats.sort_values("total", ascending=False).head(20)
axes[1].bar(range(20), top20["stay_ratio"].values, color="green", alpha=0.7, label="驻
留")
axes[1].bar(range(20), top20["travel"]/top20["total"],
bottom=top20["stay_ratio"].values,
            color="red", alpha=0.7, label="出行")
axes[1].set_xlabel("用户排名（按点数）"); axes[1].set_ylabel("比例")
axes[1].set_title("Top20 用户驻留/出行比例")
axes[1].legend()
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_user_ratio.png"), dpi=150,
bbox_inches="tight")
plt.close()

# 图4：三阶段出行点来源对比
print("生成图4：三阶段出行点来源对比")
stage1_travel = df["is_travel_stage1"].sum()
stage2_travel = df["is_travel_stage2"].sum()
# 阶段3出行 = 候选驻留点中被判为出行的
stage3_travel = ((~df["is_travel_stage12"]) & (df["cluster_id"] >= 0) & (df["label"] ==
1)).sum()
no_cluster_travel = ((~df["is_travel_stage12"]) & (df["cluster_id"] < 0)).sum()

fig, ax = plt.subplots(figsize=(10, 6))
sources = ["阶段1\n(速度>8)", "阶段2\n(单点成簇)", "阶段3\n(规则判定)", "无簇标签\n(默
认出行)"]
counts = [stage1_travel, stage2_travel, stage3_travel, no_cluster_travel]
colors = ["blue", "orange", "red", "gray"]
bars = ax.bar(sources, counts, color=colors, edgecolor="black")
for bar, count in zip(bars, counts):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
            str(count), ha="center", fontweight="bold")
ax.set_ylabel("出行点数")
ax.set_title(f"出行点来源对比（总出行={len(travel_df)}）")
plt.tight_layout()
```

```
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_travel_source.png"), dpi=150,
bbox_inches="tight")
plt.close()

print(f"\n图片保存在: {FIG_DIR}")
print(f"  fig_stage3_spatial.png")
print(f"  fig_stage3_temporal.png")
print(f"  fig_stage3_user_ratio.png")
print(f"  fig_stage3_travel_source.png")

# 统计摘要
print(f"\n{'='*60}")
print("阶段3结果统计摘要")
print(f"{'='*60}")
print(f"总点数: {len(df)}")
print(f"驻留点: {len(stay_df)} ({len(stay_df)/len(df)*100:.1f}%)")
print(f"出行点: {len(travel_df)} ({len(travel_df)/len(df)*100:.1f}%)")
print(f"出行点来源: 阶段1={stage1_travel}, 阶段2={stage2_travel}, "
      f"阶段3={stage3_travel}, 无簇标签={no_cluster_travel}")
print(f"用户平均驻留比例: {user_stats['stay_ratio'].mean()*100:.1f}%")
print(f"驻留高峰时段: {hour_stay.idxmax()}时 ({hour_stay.max()}点)")
print(f"出行高峰时段: {hour_travel.idxmax()}时 ({hour_travel.max()}点)")
```

7.4 运行结果与解读

生成4张图：空间分布、时间分布、用户驻留比例、出行点来源对比。

7.5 当堂练习

1. 查看4张图，分析驻留和出行的时空特征；
2. 思考：为什么出行高峰在早晚高峰？（提示：用户通勤）

7.6 课时小结

- 驻留点集中在驻留区域，出行点沿交通干道；
- 驻留高峰在白天和晚间，出行高峰在早晚通勤时段；
- 用户驻留比例分布不均，大部分用户驻留比例>70%；
- 出行点来源：阶段1最多，阶段3次之，阶段2和无簇标签较少。

课时8：三阶段算法总结与效果评估

8.1 课时目标

总结三阶段算法的设计思路，评估算法效果，讨论改进方向。

8.2 知识点深度讲解

8.2.1 三阶段算法设计思路

```
阶段1（粗筛）：速度>8km/h → 出行
  快速筛出明显在移动的点，计算简单（速度=距离/时间）

阶段2（聚类）：k-means + 单点成簇 → 出行
  对非出行点聚类，发现驻留区域；单点成簇的点不像是驻留

阶段3（精判）：距离<1000m 且 时间<2h → 驻留
  对候选驻留点用业务规则精判，区分真正的驻留和出行
```

设计原则：由粗到精，逐步缩小范围。
- 阶段1快速筛出明显出行点（减少后续计算量）；
- 阶段2发现驻留区域（聚类）；
- 阶段3精细判定（业务规则）。

8.2.2 算法效果评估

评估维度：
1. **驻留/出行比例**：合理范围（驻留70%-85%，出行15%-30%）；
2. **空间分布**：驻留点集中在驻留区域，出行点沿交通干道；
3. **时间分布**：驻留高峰在白天/晚间，出行高峰在早晚通勤；
4. **用户分布**：大部分用户驻留比例>70%；
5. **业务合理性**：结果符合常识（家、公司、通勤）。

8.2.3 算法局限性

1. **速度阈值固定**：8km/h 对所有用户相同，可能不适合步行慢的用户；
2. **k-means 假设球形簇**：驻留区域可能不是球形；
3. **规则阈值固定**：1000m和2h对所有用户相同；
4. **尾点全部判为出行**：可能误判（用户可能在尾点后仍在驻留区域，只是没有信令记录）；
5. **无真实标签验证**：无法计算准确率、召回率等指标。

8.2.4 改进方向

1. **自适应阈值**：根据用户的速度分布自动调整速度阈值；
2. **DBSCAN 聚类**：不需要指定K，能发现任意形状的簇；
3. **时间序列模型**：用 HMM 或 LSTM 建模用户的移动模式；
4. **多源数据融合**：结合APP使用、通话记录等数据；
5. **半监督学习**：用少量标注数据优化算法参数。

8.3 完整代码（三阶段算法总结）

```
# ============================================================
# 课时8：三阶段算法总结与效果评估
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取最终结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_full.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})

print("=" * 60)
print("三阶段算法总结与效果评估")
print("=" * 60)

# 整体效果
print(f"\n【整体效果】")
print(f"  总点数: {len(df)}")
print(f"  用户数: {df['imsi'].nunique()}")
print(f"  驻留点: {(df['label']==0).sum()} ({(df['label']==0).mean()*100:.1f}%)")
print(f"  出行点: {(df['label']==1).sum()} ({(df['label']==1).mean()*100:.1f}%)")

# 各阶段效果
print(f"\n【各阶段效果】")
print(f"  阶段1出行点(速度>8): {df['is_travel_stage1'].sum()} "
      f"({df['is_travel_stage1'].mean()*100:.1f}%)")
print(f"  阶段2出行点(单点成簇): {df['is_travel_stage2'].sum()} "
      f"({df['is_travel_stage2'].mean()*100:.1f}%)")
stage3_travel = ((~df['is_travel_stage12']) & (df['cluster_id']>=0) &
(df['label']==1)).sum()
print(f"  阶段3出行点(规则判定): {stage3_travel} ({stage3_travel/len(df)*100:.1f}%)")
```

```
# 用户级效果
user_stats = df.groupby("imsi")["label"].agg(["count", "sum"])
user_stats.columns = ["total", "travel"]
user_stats["stay"] = user_stats["total"] - user_stats["travel"]
user_stats["stay_ratio"] = user_stats["stay"] / user_stats["total"]

print(f"\n【用户级效果】")
print(f"  平均驻留比例: {user_stats['stay_ratio'].mean()*100:.1f}%")
print(f"  驻留比例中位数: {user_stats['stay_ratio'].median()*100:.1f}%")
print(f"  驻留比例>80%的用户: {(user_stats['stay_ratio']>0.8).sum()} "
      f"({(user_stats['stay_ratio']>0.8).mean()*100:.1f}%)")
print(f"  驻留比例<50%的用户: {(user_stats['stay_ratio']<0.5).sum()} "
      f"({(user_stats['stay_ratio']<0.5).mean()*100:.1f}%)")

# 时间效果
df["hour"] = df["time"].str[8:10].astype(int)
stay_hour = df[df["label"]==0].groupby("hour").size()
travel_hour = df[df["label"]==1].groupby("hour").size()
print(f"\n【时间效果】")
print(f"  驻留Top3时段: {stay_hour.nlargest(3).index.tolist()}时")
print(f"  出行Top3时段: {travel_hour.nlargest(3).index.tolist()}时")

# 算法总结
print(f"\n{'='*60}")
print("【三阶段算法总结】")
print(f"{'='*60}")
print("""
阶段1（速度粗筛）:
  - 输入：清洗后数据
  - 方法：速度=距离/时间，>8km/h判出行
  - 输出：阶段1出行点 + 候选驻留点
  - 特点：快速、简单，筛出明显出行点

阶段2（k-means聚类）:
  - 输入：候选驻留点
  - 方法：k-means聚类(K=2~10，邓恩指数选最优)，单点成簇判出行
  - 输出：聚类结果 + 阶段2出行点
  - 特点：发现驻留区域，补充阶段1漏判
```

```
阶段3（规则精判）：
  - 输入：候选驻留点（含簇标签）
  - 方法：距簇中心<1000m 且 与下点时间<2h → 驻留，否则出行
  - 输出：最终标签(0驻留/1出行)
  - 特点：精细判定，业务规则驱动
""")

print("【算法设计原则】")
print("  1. 由粗到精：阶段1粗筛→阶段2聚类→阶段3精判")
print("  2. 逐步缩小：每阶段减少候选范围，降低计算量")
print("  3. 多维度结合：速度(阶段1) + 空间(阶段2) + 时空(阶段3)")
print("  4. 业务驱动：阈值和规则基于通信业务常识")

print("\n【算法局限性】")
print("  1. 阈值固定：8km/h、1000m、2h对所有用户相同")
print("  2. k-means假设球形簇：驻留区域可能不是球形")
print("  3. 尾点全部判出行：可能误判")
print("  4. 无真实标签验证：无法计算准确率")

print("\n【改进方向】")
print("  1. 自适应阈值：根据用户速度分布自动调整")
print("  2. DBSCAN聚类：不需要指定K，任意形状簇")
print("  3. 时间序列模型：HMM/LSTM建模移动模式")
print("  4. 多源数据融合：APP、通话记录等")
print("  5. 半监督学习：少量标注数据优化参数")

print("\n" + "=" * 60)
print("出行驻留识别（三阶段）完成！")
print("=" * 60)
```

8.4 运行结果与解读

运行后展示完整的三阶段算法总结和效果评估。

8.5 当堂练习

1. 思考：如果让你改进这个算法，你会从哪里入手？
2. 尝试实现"自适应速度阈值"（根据用户速度分布的分位数确定阈值）。

8.6 课时小结

- 三阶段算法：速度粗筛→k-means聚类→规则精判，由粗到精；
- 设计原则：逐步缩小范围、多维度结合、业务驱动；
- 效果：驻留70%-85%，出行15%-30%，符合业务常识；
- 局限性：阈值固定、k-means假设球形、尾点全判出行、无真实标签；
- 改进方向：自适应阈值、DBSCAN、时间序列模型、多源融合、半监督学习。

三、本日小结

1. **阶段3算法总览**：对候选驻留点用业务规则精判，距离<1000m且时间<2h→驻留；
2. **按imsi-簇ID分组**：每组代表一个驻留区域，组内按时间排序；
3. **首/中/尾点识别**：首点(index=0)、中点、尾点(index=n-1)，尾点无下点→出行；
4. **距簇中心距离判定**：Haversine公式，1000m阈值，大部分点<1000m；
5. **时间跨度判定**：shift(-1)计算，2h阈值，大部分点<2h；
6. **最终标签判定**：两个条件同时满足→驻留(0)，否则→出行(1)；
7. **tr_stage3.py**：完整实现阶段3，输出stage3_label.csv；
8. **结果分析**：驻留70%-85%，出行15%-30%，符合业务常识；
9. **三阶段总结**：速度粗筛→k-means聚类→规则精判，由粗到精，多维度结合。

四、本日作业

1. 运行 tr_stage3.py，记录最终驻留/出行比例和各阶段出行点数量；
2. 运行可视化代码，查看4张图，写一段分析（驻留/出行的时空特征）；
3. 思考：三阶段算法中哪个阶段对最终结果影响最大？为什么？
4. 预习 Day07：思考"如何从驻留段中识别出行目的（居住/购物/旅游）？"